Evasion of Deep Learning Malware Detection via Adversarial Selective Obfuscation

Claudia Greco*, Michele Ianni†, Antonella Guzzo† and Giancarlo Fortino†
*Department of Mechanical, Energy and Management Engineering
University of Calabria, Italy
†Department of Computer Engineering, Modeling, Electronics and Systems
University of Calabria, Italy
Email: {claudia.greco, michele.ianni, antonella.guzzo, giancarlo.fortino}@unical.it,

2025 IEEE International Conference on Cyber Security and Resilience (CSR) | 979-8-3315-3591-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CSR64739.2025.11130150

***Abstract*—In this work, we present a novel approach for generating adversarial attacks on malware classification systems that rely on image-based representations of binary executables. Our method selectively applies obfuscation techniques to modify specific bytes in the binary, which correspond to adversarially perturbed pixels in the representation of malware as an image. By leveraging syntactic obfuscation strategies, we are able to transform the malware binary without compromising its functionality. Our results demonstrate that our approach effectively fools the CNN-based detection techniques, leading to misclassification. Additionally, we address the challenges associated with selective obfuscation, particularly when modified bytes map to non-instructional regions or structural elements of the binary. Overall, this research opens new avenues for understanding and defending against adversarial attacks in malware detection systems.**

***Index Terms*—malware, obfuscation, adversarial attacks, deep learning, binary analysis.**

I. Introduction

The rapid evolution of malware and its increasing sophistication have driven researchers and practitioners to explore new methods for malware detection. In recent years, deep learning techniques have gained considerable attention as a potential replacement for traditional malware detection methods, such as signature-based scanning, heuristic-based analysis, and behavior-based approaches. Unlike traditional methods, which are often built around the semantic meaning of code and behaviors associated with malicious intent, deep learning-based approaches focus on learning abstract features from data. This has led to the emergence of solutions that attempt to replace human-driven, semantic-based detection techniques with automated models, including Convolutional Neural Networks (CNNs).

Numerous deep learning-based methods for malware detection have been proposed, many of which report impressive results in terms of classification accuracy. These models, however, are increasingly disconnected from the semantic aspects of malware behavior, relying instead on structural patterns found in the data. Traditional methods, such as signature-based detection, rely on identifying specific code patterns and behaviors that are inherently tied to the malicious actions of the software. By contrast, CNN-based models treat the binary structure of a file as input data, converting the byte sequences into images or similar formats and applying image classification techniques. While such methods often achieve high accuracy scores, the reliance on structural features rather than behavior introduces fundamental limitations, which this paper aims to explore.

Moreover, even when explainability techniques are applied to these models, the features highlighted by the explainable artificial intelligence (XAI) tools often lack a meaningful connection to the malicious behavior of the software. For instance, in CNN-based models, the features under scrutiny are typically individual pixels or image regions that correspond to byte sequences, but these are removed from the behavioral semantics that are critical in distinguishing malicious software. This makes the interpretation of the model's decision less useful for practitioners in the cybersecurity domain, as it does not directly correlate with the actual nature of the threat.

To leverage pre-trained models and reduce training costs, several recent approaches have adopted transfer learning. One popular method involves using pre-trained image classification models, such as ResNet, and applying them to malware detection by representing executable binaries as images. This representation converts the bytecode into grayscale pixel values, which are then classified using CNNs as either benign or malicious, or even further into specific malware families. While these approaches have demonstrated high levels of accuracy, the focus on structural features, rather than behavioral characteristics, raises questions about the robustness and reliability of such detectors.

In this paper, we explore the limitations of deep learning-based malware detection systems by applying adversarial attacks. Specifically, we use selective syntactic obfuscation to modify small portions of the malware, leaving its overall semantics and functionality unchanged, while simultaneously causing the model to misclassify the malware. Our approach minimizes the changes required to fool the model, which is crucial because extensive modifications to the binary (such as widespread obfuscation or packing) can themselves be indicators of malicious intent [1], [2], as seen in polymorphic and metamorphic malware families. By highlighting the ease with which such models can be bypassed through targeted obfuscation, we argue for a rethinking of how deep learning models should be integrated into malware detection workflows.

The main contributions of this paper are as follows:

- We analyze the limitations of CNN-based malware detection systems by demonstrating how their reliance on structural features, rather than behavioral semantics, makes them vulnerable to adversarial attacks.
- We propose a novel adversarial attack strategy that involves selective syntactic obfuscation of the malware code, which preserves the malicious functionality while causing the model to misclassify the malware.
- We demonstrate that this targeted obfuscation approach is highly effective, even with minimal changes to the binary, unlike traditional obfuscation techniques that significantly modify the binary and can trigger other detection methods.

This work provides important insights into the vulnerabilities of machine learning-based malware detection systems and suggests potential pathways for developing more robust and semantically aware models in the future.

II. Background and Related Work

A. Malware Classification Using Images

In recent years, the use of image-based representations for malware detection has gained traction, motivated by the ability of deep learning models—especially CNNs—to recognize patterns in visual data. Several approaches have been proposed to convert binary executables into images.

In [3], the authors introduce the use of Self-Organizing Maps to visualize and detect viruses. A significant advancement in the field came from Nataraj et al. [4], who proposed the visualization of malware binaries as gray-scale images. Their method used image processing techniques to classify malware, combining this visual representation with a K-nearest neighbor classifier using Euclidean distance for effective malware classification. In [5] the authors also explored gray-scale image-based techniques, converting executables into "byteplots" and employing Support Vector Machines to classify malware, achieving a 95% accuracy rate on a large dataset. In [6] the authors combined visualized images with entropy graphs to detect and classify both new and variant malware. In [7] the authors focused on disassembling binary executables into opcode sequences, which were subsequently converted into images for classification using CNNs, further advancing the intersection of image processing and malware detection. In [8] a new algorithm is proposed, which converts the disassembled malware codes into gray images based on SimHash and then identifies their families by a CNN.

These visualizations allow researchers to exploit the pattern recognition capabilities of CNNs, which excel at distinguishing between visual features. By using pre-trained models like ResNet through transfer learning, researchers can classify malware samples based on these visual representations, significantly reducing the computational resources required for training. This transfer learning technique allows for effective use of CNN architectures, which were originally trained on large image datasets like ImageNet, to classify malware.

B. Adversarial Attacks on Image Classification Models

Adversarial attacks target machine learning models by introducing small, carefully crafted perturbations to input data, causing the model to misclassify the input. These attacks pose a serious threat to deep learning models, especially in the context of image classification. Despite the perturbations being almost imperceptible to human observers, they can lead to significant errors in model predictions. The ability of adversarial attacks to fool models highlights vulnerabilities in many neural networks, including those used for malware detection.

Several attack methods exist, each with different objectives, such as minimizing the number of changed pixels or controlling the magnitude of the perturbations. These perturbations are typically measured using different norms that quantify the distance between the original input and the adversarial example:

- $\mathbf{L_0}$ **norm**: Measures the number of altered pixels, regardless of how much each pixel is changed. This is useful for attacks that aim to minimize the number of modifications.
- $\mathbf{L_2}$ **norm**: Measures the Euclidean distance between the original and perturbed images, taking into account both the number of changes and their magnitude.
- $\mathbf{L_\infty}$ **norm**: Focuses on the largest individual pixel change, limiting the maximum change to any one pixel while allowing many small changes.

One of the earliest and most well-known attacks is the *Fast Gradient Sign Method* (FGSM), introduced by Goodfellow et al. [9]. FGSM generates adversarial examples by perturbing the image in the direction of the gradient of the loss function with respect to the input. This results in small but effective modifications under the L_∞ norm. FGSM is fast and computationally efficient, making it a widely studied attack.

Another powerful attack is the *Carlini & Wagner* (C&W) attack [10], which optimizes a targeted adversarial example using a customized objective function. This method is effective in generating adversarial examples under various norms (typically L_2 or L_∞), and is known for its ability to bypass many state-of-the-art defense mechanisms.

Projected Gradient Descent (PGD) [11] extends FGSM by iterating over multiple steps of gradient-based updates, refining the perturbations to achieve better attack success. PGD has become the gold standard for evaluating the robustness of models due to its efficacy in generating strong adversarial examples.

Gradient-free approaches like *HopSkipJump* [12] operate in a black-box setting, where the attacker does not have access to the model's gradients but can query the model for outputs. HopSkipJump reduces the number of modifications, focusing on minimizing the L_0 norm, making it particularly useful when the number of pixel changes needs to be limited.

In our research, we prioritize minimizing the number of changes (L_0 norm) while ensuring the malware binary maintains its functionality. This approach aligns with our goal of evading detection by making minimal modifications that

are not easily detectable by traditional signature-based or behavioral analysis methods. To achieve this, we employ the *Jacobian-based Saliency Map Attack* (JSMA) [13]. JSMA selectively modifies the most influential pixels in the image based on a saliency map, which identifies which pixels have the greatest impact on the model's classification decision. We chose JSMA for its ability to minimize the number of modified pixels while effectively evading detection, but we will provide more detail on this method in the next section.

C. Jacobian-based Saliency Map Attack (JSMA)

The Jacobian-based Saliency Map Attack (JSMA) is an adversarial attack designed to misclassify inputs by minimally perturbing the input features. In the context of image-based classification models, the features correspond to the pixels of the image. The goal of JSMA is to perturb an input feature vector $\mathbf{x}$ to produce an adversarial sample $\mathbf{x}^* = \mathbf{x} + \delta\mathbf{x}$, where $\delta\mathbf{x}$ is the perturbation vector. This perturbation is designed to be small and subtle, yet sufficient to cause a misclassification, i.e.,

$$\arg\min_{\delta\mathbf{x}} ||\delta\mathbf{x}|| \quad \text{s.t.} \quad f(\mathbf{x}^*) = y^* \neq y \tag{1}$$

The idea behind JSMA is to use adversarial saliency maps to determine which features (pixels) to perturb in order to mislead the neural network.

1) The Neural Network Output: Let the neural network output a vector $\mathbf{f}(\mathbf{x})$, where each element $f_j(\mathbf{x})$ represents the score (logit) for class j before applying the softmax function. The predicted label of the input $\mathbf{x}$ is the class with the maximum score:

$$\text{label}(\mathbf{x}) = \arg\max_j f_j(\mathbf{x}) \tag{2}$$

Given a target class $t \neq \text{label}(\mathbf{x})$, the goal is to modify $\mathbf{x}$ so that the neural network misclassifies it as t. To achieve this, we aim to increase the score $f_t(\mathbf{x})$ of the target class while decreasing the scores $f_j(\mathbf{x})$ for all other classes $j \neq t$.

To determine which features (pixels) to modify, we analyze the gradient of the output with respect to the input features. Specifically, we compute the Jacobian matrix:

$$J_{ij}(\mathbf{x}) := \frac{\partial f_j(\mathbf{x})}{\partial x_i} \tag{3}$$

Here, $J_{ij}(\mathbf{x})$ represents the partial derivative of the score for class j with respect to input feature x_i. For a model with n classes, this requires n rounds of backpropagation, as we compute the gradient of the score for each class with respect to every input feature.

2) Adversarial Saliency Map: The adversarial saliency map $s(i; \mathbf{x}, t)$ is used to guide which pixels to modify. The saliency map is constructed based on the gradients $J_{ij}(\mathbf{x})$ in such a way that it identifies the pixels whose modification would most likely result in misclassification. We want to modify pixels x_i that will increase $f_t(\mathbf{x})$ while minimizing increases to $f_j(\mathbf{x})$ for $j \neq t$. The saliency map is defined as:

$$s(i; \mathbf{x}, t) := 0 \quad \text{if} \quad J_{it}(\mathbf{x}) < 0 \quad \text{or} \quad \sum_{j \neq t} J_{ij}(\mathbf{x}) > 0 \tag{4}$$

In other words, if increasing x_i would decrease $f_t(\mathbf{x})$ or increase the score of any other class $j \neq t$, then we do not modify x_i. Otherwise, the saliency map is computed as:

$$s(i; \mathbf{x}, t) := J_{it}(\mathbf{x}) \cdot \left| \sum_{j \neq t} J_{ij}(\mathbf{x}) \right| \tag{5}$$

This ensures that pixels are selected for modification only if they will increase the target score $f_t(\mathbf{x})$ and decrease the scores for all other classes. The magnitude of the change for each selected pixel is proportional to the value of $s(i; \mathbf{x}, t)$.

3) Saliency Map Construction and Perturbation: Once the saliency map is computed, the adversarial perturbation is applied to the pixel(s) with the highest saliency values, i.e., the features that most significantly influence the classification decision. This process is repeated iteratively until the desired misclassification is achieved, with the goal of minimizing the number of modified pixels while ensuring the adversarial success. The result is an adversarial sample $\mathbf{x}^*$, which appears visually similar to the original sample $\mathbf{x}$, but is misclassified by the model.

The algorithm is iterative: at each step, it computes the saliency map based on the Jacobian matrix of the network's output with respect to the input features. It then modifies the most relevant features (those contributing the most to misclassification) while ensuring the perturbations remain minimal. The procedure stops when the network classifies the input into the target class or when the magnitude of the perturbation reaches a threshold.

Algorithm 1 Crafting Adversarial Samples using JSMA

Input: $\mathbf{x}$: Input sample, y^*: Target class, $\mathbf{F}$: Neural network function, Υ: Max perturbation, θ: Step size
Output: $\mathbf{x}^*$: Adversarial sample
Initialize: $\mathbf{x}^* \leftarrow \mathbf{x}$, $\delta\mathbf{x} \leftarrow \mathbf{0}$, $\Gamma \leftarrow \{1, \ldots, |\mathbf{x}|\}$
while $\mathbf{F}(\mathbf{x}^*) \neq y^*$ **and** $\|\delta\mathbf{x}\| < \Upsilon$ **do**
  Compute forward derivative $J_{\mathbf{F}}(\mathbf{x}^*)$
  Compute saliency map:
    $S(\mathbf{x}, y^*) = \text{saliency\_map}(J_{\mathbf{F}}(\mathbf{x}^*), \Gamma, y^*)$
  $i_{\text{max}} \leftarrow \arg\max_i S(\mathbf{x}, y^*)[i]$
  Modify $\mathbf{x}^*_{i_{\text{max}}}$ by θ
  Update $\delta\mathbf{x} \leftarrow \mathbf{x}^* - \mathbf{x}$
end while
return $\mathbf{x}^*$

III. Proposal and Experimental Evaluation

In this section, we investigate the effectiveness and limitations of image-based malware detection approaches that use CNNs. As a conversion mechanism we use, as an example, the method proposed by Nataraj et al. [4], which converts binary executables into gray-scale images for classification and then

we leverage a CNN. The conversion process involves reading the binary data as a sequence of bytes, where each byte is treated as a pixel with an intensity value in the range [0-255]. These bytes are then arranged into a 2D image, forming a gray-scale representation of the malware. CNNs are trained on these images to classify various types of malware. While this technique has shown considerable promise in detecting malware, our objective is to investigate its susceptibility to adversarial attacks. Specifically, we aim to test the robustness of CNNs when adversarial perturbations are introduced into the malware images, with the goal of evading detection.

A. Adversarial Attack Application and Methodology

For our experiments, we apply the JSMA technique to perturb malware images that have been correctly classified by the CNN. Our primary goal is to cause the CNN to misclassify these images while minimizing the number of pixel modifications. This focus on minimizing the number of pixel changes, rather than the magnitude of the perturbations, is crucial in the context of executable binaries, as even slight modifications at the byte level could lead to a loss of functionality. The adversarial attack is crafted to ensure that the changes in the image representation of the binary are minimal, allowing us to maintain the executable's original behavior.

The general methodology for applying JSMA to malware images involves the following steps:

- **Step 1: Model Training**
  We begin by training a CNN model on the dataset of malware and benign binaries, using the image representation described in [4]. The model is trained on 64×64 gray-scale images, which represent the original binary data as pixels. The CNN architecture consists of convolutional layers followed by fully connected layers for classification. The model is trained until it achieves satisfactory performance, with a final accuracy of 0.957 on the test set.
- **Step 2: Adversarial Example Generation**
  Once the model has been trained, we apply the JSMA to generate adversarial examples. The attack selects a subset of pixels from the original malware image based on the saliency map, which identifies the pixels most likely to influence the model's prediction. These pixels are then modified in a targeted manner, with the objective of causing the CNN to incorrectly classify the malware image as benign.
- **Step 3: Minimizing Pixel Modifications**
  A key aspect of our approach is to minimize the number of pixel modifications required to fool the detector. This constraint is important because, in the context of binary executables, each pixel corresponds to a byte, and altering too many bytes may render the binary non-functional. Our implementation of JSMA prioritizes changes to pixels that have the highest impact on the classification outcome, allowing us to achieve misclassification with a minimal number of modifications.
- **Step 4: Binary Functionality Preservation**
  After generating the adversarial example, we map the modified pixels back to the corresponding bytes in the binary executable and we transform the corresponding bytes using the techniques described in the next subsection.

B. Selective Obfuscation of Executable Binaries

Once the pixels that have been modified by the adversarial attack on the malware image are identified, the next step is to map these pixels back to their corresponding bytes in the original binary executable. The goal of selective obfuscation is to modify only those bytes that correspond to the changed pixels, while preserving the overall functionality of the binary. This process is referred to as *selective* because it targets specific portions of the binary rather than applying global obfuscation techniques.

To achieve this, we utilize a set of syntactic obfuscation techniques that modify the code without affecting its semantics. These transformations are designed to change the representation of the binary in a way that alters the corresponding pixels in the image, making them closer to the adversarially perturbed pixels, while ensuring that the modified binary remains functionally equivalent to the original. The following obfuscation techniques are employed in this selective process:

- **Instruction Substitution**
  Instruction substitution involves replacing certain instructions with semantically equivalent ones. For example, instead of using the `add` instruction to increase a register value, we could use `inc` to achieve the same effect. Similarly, other arithmetic and logical operations may have multiple equivalent representations. This technique is particularly useful when the byte corresponding to a certain pixel in the image can be modified by substituting an instruction with another that results in a different byte encoding while preserving the same functionality.
- **Instruction Reordering**
  Instruction reordering takes advantage of the fact that certain instructions are independent of each other and can be executed in any order without altering the program's behavior. By swapping the order of these instructions, we can alter the byte sequence in the binary, and consequently, the pixel representation in the image. This technique is particularly effective when the bytes corresponding to the modified pixels belong to regions of the binary where instruction order is not fixed (e.g., non-dependent operations).
- **Inlining and Outlining**
  Inlining and outlining are techniques used to either expand or contract functions. Inlining replaces a function call with the actual body of the function, increasing the size of the code but altering the sequence of instructions. Outlining, on the other hand, involves moving a block of code (such as part of a function) to a separate function, which is then called from the original location. These transformations can be used to introduce variations in the byte sequence of the binary, thus modifying the corresponding pixel values in the image.
- **Register Swapping**
  Register swapping is another syntactic obfuscation tech-

nique in which the roles of different registers in the program are interchanged. For example, if one part of the program uses register `eax` and another part uses register `ebx`, these registers can be swapped, provided their values are preserved across the transformation. This technique can change the bytes encoding the instructions that reference the registers, thereby altering the image representation without modifying the semantics of the program.

1) Selective Application of Obfuscation Techniques: The process of applying these obfuscation techniques selectively is driven by the need to match the transformed bytes to the pixel values in the adversarial image. Given the byte-to-pixel mapping, the goal is to identify which instructions or structures in the binary correspond to the modified pixels and apply transformations that modify these bytes while staying as close as possible to the desired values from the adversarial image.

To achieve this, we first analyze the disassembly of the binary executable to identify the exact instructions corresponding to the modified bytes. Once these instructions are identified, we apply the most appropriate obfuscation technique to achieve a transformation.

2) Challenges in Selective Obfuscation: While the selective obfuscation approach is effective in many cases, several challenges arise that complicate the process. One of the most significant challenges occurs when the bytes corresponding to the modified pixels do not map to executable instructions, but instead to non-code regions such as data segments, headers, or metadata. For example, these regions may include addresses, constants, or section headers, which cannot be easily obfuscated without risking the integrity of the binary. Modifying such bytes could cause the program to crash, alter its input/output behavior, or make it non-executable. Another challenge arises when the bytes correspond to structural elements of the binary, such as relocation entries, symbol tables, or import/export directories. These components are essential for linking and executing the binary correctly, and any modification could lead to invalid pointers or addresses, causing the program to fail. Obfuscating these regions requires careful handling, as even small modifications can break the program. In some cases, the bytes corresponding to the modified pixels might be part of instructions for which no suitable substitution exists. For example, if the adversarial pixel suggests a byte value that cannot be achieved through any known instruction substitution, this leaves us with limited options for transformation. Furthermore, certain instructions may involve immediate values or addressing modes that are tightly constrained, making it difficult to achieve a byte encoding that matches the desired value from the adversarial image. The mapping between bytes in the binary and pixels in the image is straightforward in terms of position, but not in terms of the relationship between the byte values and pixel intensities. The adversarial image may suggest certain pixel intensities that are difficult or impossible to replicate through simple obfuscation, especially when the corresponding bytes are part of a multi-byte instruction or a sequence of dependent instructions. This complexity further complicates the selective obfuscation process.

Due to these challenges, there may be cases where traditional obfuscation techniques fail to achieve the desired byte modifications. In order to evade detection, our strategy is to first apply syntactic obfuscation techniques, as described earlier, to selectively modify the bytes corresponding to the adversarially perturbed pixels. If these techniques are successful and lead to misclassification by the CNN, no further action is needed. However, when obfuscation techniques fail due to structural constraints, non-instructional bytes, or other challenges, we attempt to skip over the difficult-to-modify bytes and focus on transforming those that can be safely obfuscated. If misclassification is still not achieved at this point, as last resort, we employ a more direct and effective solution: XOR-based byte manipulation.

C. XOR-based Byte Manipulation

This technique, while simplistic, provides a powerful method for directly altering the bytes of the binary to match the desired values derived from the adversarial image. The core idea behind this approach is to introduce an additional routine into the binary that performs XOR operations on the bytes identified for transformation. By carefully choosing the XOR keys, we can modify any byte in the binary to match the corresponding byte in the adversarial image. This is accomplished by applying the XOR operation at runtime, ensuring that the bytes are transformed dynamically without altering the overall structure or execution flow of the binary.

The XOR-based manipulation is implemented as follows: once the bytes that need to be changed have been identified, we create a simple mapping that associates the offsets of these bytes in the binary with the corresponding XOR keys. The XOR key for each byte is calculated as the difference between the original byte value and the desired byte value, i.e., the byte value that corresponds to the pixel intensity in the adversarial image. This mapping is stored in a data structure within the binary. An additional routine is added to the binary, which iterates through the mapping and applies the XOR operation to each specified byte. The result of this operation is that the original bytes are transformed into the desired byte values. This routine ensures that the necessary transformations are applied only to the targeted bytes, leaving the rest of the binary untouched. The XOR-based approach is extremely flexible, as it allows for arbitrary changes to any byte in the binary, regardless of whether it corresponds to an instruction, data, or structural element making it an ideal fallback solution when other methods fail.

D. Experimental Setup

We used a dataset[1] composed of binaries from different malware families as well as benign executables, which were converted into gray-scale images using the method proposed by Nataraj et al. The image dimensions were determined based

[1]https://paperswithcode.com/dataset/malimg

on the size of each binary file, with each byte mapped to a corresponding pixel in the image. For our CNN model, we employed an architecture consisting of several convolutional layers followed by fully connected layers and a softmax output layer for classification. The model was trained using a standard cross-entropy loss function and optimized using the Adam optimizer. The model achieved a final accuracy of 0.957 on the test set. Once the model achieved satisfactory accuracy on the test set, we applied JSMA to generate adversarial examples from a subset of malware images. The attack was configured to target a generic misclassification, seeking to modify as few pixels as possible while ensuring that the adversarial examples were misclassified.

E. Results and Discussion

Our experimental evaluation demonstrated the vulnerability of CNN-based malware detection systems to adversarial attacks. We chosen 10 different malware and we were able to make the model misclassify all of them. It is important to notice that only a small number of pixels were modified (the average is 5%), confirming that even slight perturbations can be sufficient to deceive the CNN. Additionally, our results highlight the importance of understanding how CNNs interpret image-based representations of malware. The reliance on image features, rather than direct code analysis, makes these models susceptible to adversarial perturbations that do not necessarily reflect meaningful changes at the semantic level of the malware.

IV. Conclusion and Future work

In this paper, we explored a selective obfuscation approach for generating adversarial examples that target CNN-based malware classifiers. By identifying the pixels modified in an adversarial image and mapping them back to corresponding bytes in the malware binary, we selectively applied obfuscation techniques to these specific regions. This selective approach ensures that we modify the binary in a controlled and targeted manner, preserving its overall functionality while altering its image representation to induce misclassification.

Despite the promising results, several challenges and opportunities for improvement remain. One of the key challenges lies in identifying appropriate transformations for complex regions of the binary that do not correspond to executable instructions. Additionally, while the XOR-based manipulation method offers flexibility, its implementation can increase the binary's susceptibility to detection by more advanced analysis tools. In future work, we aim to explore several directions to enhance the robustness and effectiveness of this approach. First, we plan to investigate new adversarial attack techniques tailored to image-based malware detection systems. Second, we will seek to develop new obfuscation mechanisms that offer greater flexibility in transforming binary instructions while minimizing the risk of detection. Finally, we intend to automate the process of selective obfuscation, enabling more efficient and scalable generation of adversarial examples. This automation will reduce the manual effort involved in identifying and transforming relevant bytes, leading to more streamlined adversarial attack generation and opening new possibilities for real-time obfuscation in practical scenarios.

Acknowledgments

This work was partially supported by projects i) SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union - NextGenerationEU and ii) the Next Generation EU - Italian NRRP, Mission 4, Component 2, Investment 1.5, call for the creation and strenghtening of "Innovation Ecosystems", building "Territorial R&D Leaders" (Directorial Decree n. 2021/3277) - Project Tech4You - Technologies for climate change adaptation and quality of life improvement, n. ECS0000009.

References

[1] C. Greco, M. Ianni, A. Guzzo, and G. Fortino, "Enabling obfuscation detection in binary software through explainable ai," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–12, 2024.

[2] ——, "Explaining binary obfuscation," in *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, July 2023, pp. 22–27.

[3] I. Yoo, "Visualizing windows executable viruses using self-organizing maps," in *Proceedings of the 2004 ACM workshop on Visualization and data mining for computer security*, 2004, pp. 82–89.

[4] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: visualization and automatic classification," in *Proceedings of the 8th international symposium on visualization for cyber security*, 2011, pp. 1–7.

[5] K. Kancherla and S. Mukkamala, "Image visualization based malware detection," in *2013 IEEE symposium on computational intelligence in cyber security (CICS)*. IEEE, 2013, pp. 40–44.

[6] K. S. Han, J. H. Lim, B. Kang, and E. G. Im, "Malware analysis using visualized images and entropy graphs," *International Journal of Information Security*, vol. 14, pp. 1–14, 2015.

[7] J. Zhang, Z. Qin, H. Yin, L. Ou, and Y. Hu, "Irmd: malware variant detection using opcode image recognition," in *2016 IEEE 22nd International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2016, pp. 1175–1180.

[8] S. Ni, Q. Qian, and R. Zhang, "Malware identification using visualization images and deep learning," *Computers & Security*, vol. 77, pp. 871–885, 2018.

[9] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[10] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 ieee symposium on security and privacy (sp)*. Ieee, 2017, pp. 39–57.

[11] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.

[12] J. Chen, M. I. Jordan, and M. J. Wainwright, "Hopskipjumpattack: A query-efficient decision-based attack," in *2020 ieee symposium on security and privacy (sp)*. IEEE, 2020, pp. 1277–1294.

[13] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *2016 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 2016, pp. 372–387.